Session 2.6

Architecture as a Constraint on W, and What More Parameters Buy

Restrictions that encode what we know about the input, and the opposite move: more parameters than observations



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

The Same Layer, with Restrictions

- Every network so far has been **dense**: every entry of every W free, nothing assumed about the input.
- A dense layer is indifferent to its input's structure. Shuffle the pixels of every digit the same way and it learns exactly as well — which means it never used the fact that pixels have neighbours.
- **An architecture is a constraint on W**: entries fixed at zero, entries forced equal, entries forced non-negative. Each one is a claim about the target.
- Three of those claims are worth an economist's time: locality (convolutions), shape (monotone and convex policies), and order — or its absence (sequences, networks of agents).
- **Then the opposite question: what does an *un*restricted network buy from having more parameters than observations?**

© Zhigang Feng · Redistribution prohibited without permission

Session 2.6

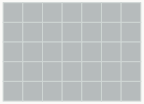
Architecture as a Constraint on W

The same layer, with restrictions that encode what we know about the input

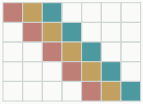


© Zhigang Feng Redistribution prohibited without permission

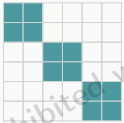
One Idea, Three Times



dense
every entry free



convolution
sparse and repeated



recurrent
one block at every date



graph
follows the links

Redistribution prohibited without permission

- A layer is $\sigma(Wx + b)$. White cells are fixed at zero; in the dense matrix every grey cell is free; in the other three, cells of one colour are *one* shared parameter.
- A dense W assumes nothing about the input, so everything must be learned from data.
- **A constraint on W is a prior about the target. True, it saves parameters and data. False, it is bias.**

Source: Miao & Feng, *AI for Economists*, §9.6.

Session 2.6

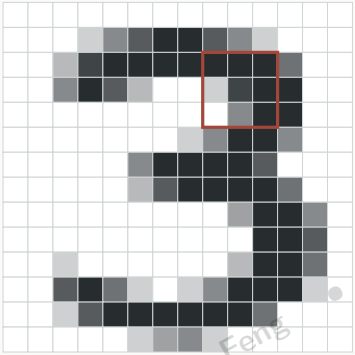
Locality: the Convolution

Two constraints on W — sparse, and repeated



© Zhigang Feng Redistribution prohibited without permission

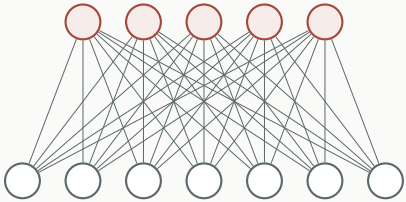
Back to the Digit: What a Dense Layer Ignores



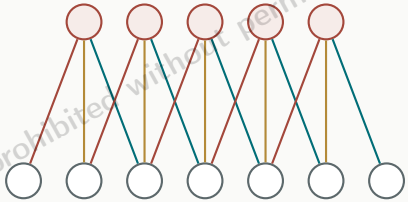
- Deck 2.1 flattened the image into a vector. A dense first layer then has one weight per pixel per hidden unit: $196 \times 128 = 25{,}088$ weights for the 14×14 digit here, $784 \times 128 = 100{,}352$ for MNIST's 28×28.
- Shuffle the pixels, the same way in every image, and a dense network learns exactly as well. It never knew which pixels were neighbours.
- Yet a stroke is a local object: the red 3×3 patch means the same thing wherever it sits.
- **Two facts about images are unused: nearby pixels belong together, and a pattern can appear anywhere.**

© Zhigang Feng
Redistribution prohibited without permission

Dense vs Convolution: 35 Weights vs 3



dense: 35 lines, 35 free weights



convolution: 15 lines, 3 free weights

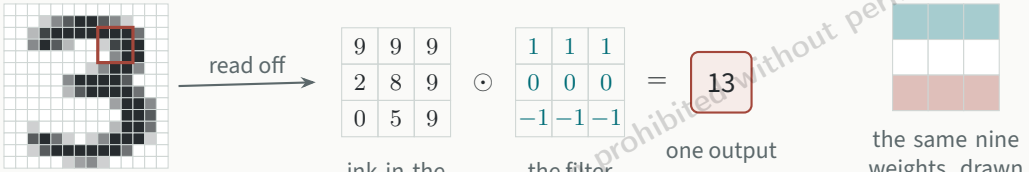
Redistribution prohibited without permission

© Yuqing Feng

- Seven inputs, five outputs. On the right each output sees three adjacent inputs (**local connection**), and lines of one colour are one parameter (**weight sharing**).
- The three shared weights are the **filter**, or **kernel**. Sliding it along the input is a **convolution**.
- **Convolution is a dense layer after two constraints on W: sparse, and repeated.**

Source: Miao & Feng, *AI for Economists*, §9.6, Fig. 9.9.

The Receptive Field

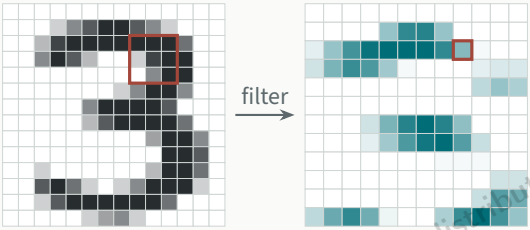


© Zhigang Feng. Redistribution prohibited without permission

- The **receptive field** of an output is the part of the input it can see: here the 3×3 patch outlined in red, and nothing else.
- The output is an inner product: $(9+9+9)-(0+5+9)=13$. The filter asks whether there is ink above and paper below, so it fires on the lower edge of a horizontal stroke.
- **Drawn as a picture it is the template neuron of Deck 2.1, shrunk to nine weights — and those nine are reused at every position on the next slide.**

Source: Patch: rows 3 to 5, columns 9 to 11 of the 14×14 digit of Deck 2.1.

Feature Maps: One Question Asked Everywhere

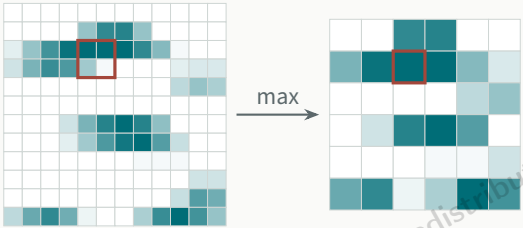


- Slide the filter over every position, apply a ReLU, and record the answers: a **feature map**.
- It lights up along the lower edges of horizontal strokes, wherever they are.
- A layer has many filters, often 32 or 64, each with its own map. They are learned, not designed.
- **One filter is one question, asked at every location with the same nine weights.**

Source: Computed from the digit of Deck 2.1 with the filter of the previous frame; the red cell is the value 13.

© Zhigang Feng Redistribution prohibited without permission

Pooling

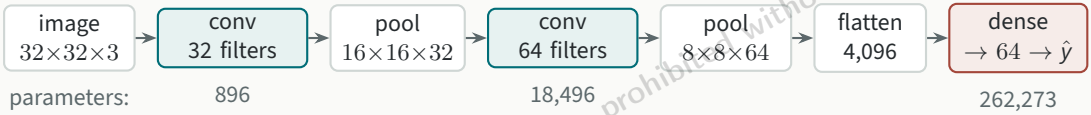


- **Pooling** replaces each small block by one summary number. **Max pooling** keeps the largest.
- The red 2×2 block holds $27,\ 27,\ 10,\ 1$; it becomes 27. Resolution halves: $12 \times 12 \to 6 \times 6$.
- The question becomes "is the edge somewhere in this block?" Pooling has no parameters.
- **Convolution extracts features; pooling reduces resolution, trading location for robustness.**

© Zhigang Feng Redistribution prohibited without permission

The Full Pipeline: Equivariance, then Approximate Invariance

A colour image now, not the digit: three channels, 32×32 pixels.



- **Equivariance**: shift the input one pixel and the feature map shifts one pixel (weight sharing).
- **Invariance**: the output does not change. Pooling gives it approximately, for small shifts.
- Of 281,665 parameters the convolutions hold 6.9%; the dense head holds 93.1%.
- **A convolutional network is a learned feature extractor followed by the network you already know.**

Source: Miao & Feng, *AI for Economists*, §9.6, Fig. 9.10.

© Tongxi Feng Redistribution prohibited without permission

What the Constraint Buys, and What It Assumes

- A 256×256 colour image is $196{,}608$ numbers: a dense layer into 256 units needs about 5×10^7 weights, while sixty-four 3×3 filters over three colour channels need $64 \times (3 \cdot 3 \cdot 3 + 1) = 1{,}792$ — four orders of magnitude fewer.
- Fewer parameters need fewer observations. In economics the sample is what it is, so this is decisive.
- The assumption bought: nearby inputs are related, distant ones are not, and a pattern means the same thing everywhere.
- Apply a convolution to data with no spatial order, such as the columns of a survey, and the constraint is simply wrong.
- Random, untrained filters with a ridge regression on top match deep training on satellite tasks (Rolf et al., 2021): the locality and the sharing do the work.
- **Parameters saved are paid for with an assumption. Check the assumption, not the parameter count.**

© Thiago Ferreira · Redistribution prohibited without permission

Source: E. Rolf et al., *Nature Communications* 12, 2021; Miao & Feng, *AI for Economists*, §9.6.

Session 2.6

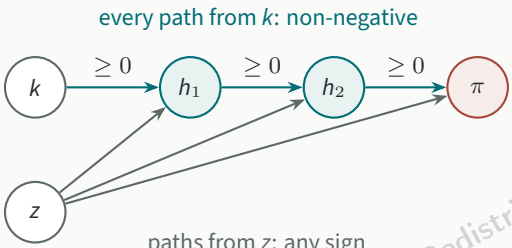
Shape You Can Guarantee

Monotone and convex by construction, and what the guarantee costs



© Zhigang Feng Redistribution prohibited without permission

Monotone Policies: Which Weights Carry the Sign

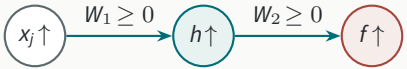


- Theory often gives the shape: saving rises in wealth, demand falls in own price.
- A **monotone network**: every weight on every path from x_j non-negative and activations non-decreasing give $\partial f / \partial x_j \geq 0$ everywhere.
- *Partial* monotonicity is the useful case: constrain the paths from k, leave those from z.
- **Impose $W \geq 0$ by training V in $W = \mathrm{softplus}(V)$; a clip stalls the gradient at the boundary.**

Source: J. Sill, "Monotonic networks", *NeurIPS* 10, 1998.

© Zhigang Feng
Redistribution prohibited without permission

Monotone Networks: Why the Sign Survives, and Where Theory Supplies It



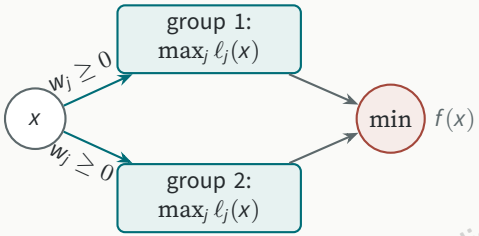
each layer: a larger input gives a larger pre-activation, and a non-decreasing σ passes it on

- **Both conditions are needed.** One negative weight on a path from x_j breaks it; so does a non-monotone activation (GELU and Swish dip below zero).
- **Decreasing instead**: feed $-x_j$, e.g. minus own price for a demand curve.
- **Where theory gives the sign**: saving $a'(a, z)$ rising in assets; consumption rising in cash on hand; demand falling in own price; a value function rising in the resource state.
- **What it prevents**: the spurious wiggles of an unconstrained fit, which a residual plot can hide.
- **Clip or reparameterise?** A weight clipped at zero gets no gradient and stays dead; $W = \text{softplus}(V)$ does not.

Source: J. Sill, *NeurIPS* 10, 1998; Miao & Feng, *AI for Economists*, ch. 9.

© Zhigang Feng. Unauthorized distribution prohibited without permission

What the Sign Constraint Costs, and What It Does Not



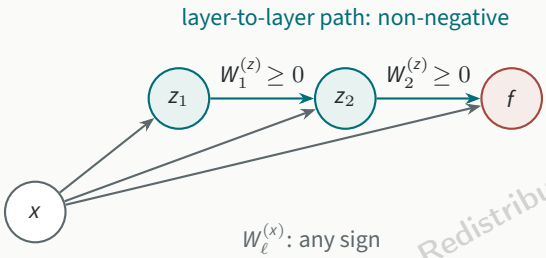
$\ell_j(x) = w_j x + b_j$:
Sill's min–max network

- Sign constraints shrink the parameter set, so a constrained network often trains more slowly and to a *higher* training loss.
- That is a harder search, not a smaller class: maxima within groups of linear units, a minimum across groups, and the family is universal among monotone functions.
- Penalising violations instead stays flexible, and holds only where it was sampled.
- **The unconstrained network reaches its lower loss partly with functions the theory forbids.**

Source: J. Sill, "Monotonic networks", *NeurIPS* 10, 1998, §2 (the min–max architecture).

© Zhigang Feng
Redistribution prohibited without permission

Input-Convex Networks: Which Weights, Which Arguments, Which Direction



- An **input-convex network**: $z_{\ell+1} = \sigma\left(W_\ell^{(z)} z_\ell + W_\ell^{(x)} x + b_\ell\right)$.
- Convex in x if $W_\ell^{(z)} \geq 0$ and σ is convex and non-decreasing (ReLU, softplus). The direct weights $W_\ell^{(x)}$ are free.
- Concave: take $-f$. Convex in some arguments only: constrain only their paths.
- **Examples: a cost function convex in input prices; a production function concave in capital.**

© Zhigang Feng
Redistribution prohibited without permission

Source: B. Amos, L. Xu, J. Z. Kolter, "Input convex neural networks", *ICML*, 2017.

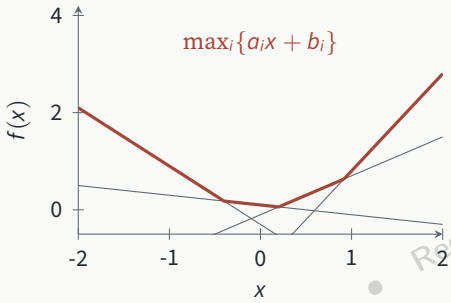
Why the Network Is Convex: Three Facts and an Induction

- **Base.** $z_0 = x$ is linear in x, hence convex.
- **Step.** Suppose every unit of z_ℓ is convex in x. Then
  - $W_\ell^{(z)} z_\ell$ is a *non-negative* combination of convex functions: convex;
  - adding the affine term $W_\ell^{(x)} x + b_\ell$, of any sign, keeps it convex;
  - a convex, non-decreasing σ of a convex function is convex.
- So every layer, and the output f, is convex in x, at every value of the parameters.
- **Both conditions carry weight.** A negative $W^{(z)}$ flips a convex unit to concave; a non-monotone σ such as $\tanh$ breaks the last step.
- **The free direct weights are why it can still fit**: each layer sees x again and can tilt and place the function.

© Zhaoning Feng · Redistribution prohibited without permission

Source: B. Amos, L. Xu, J. Z. Kolter, "Input convex neural networks", *ICML*, 2017, Prop. 1.

Does Convexity Cost Flexibility? A Shape, Not a Functional Form



- **Theorem**: an input-convex network approximates any convex function on a compact domain.
- Why: a maximum of affine functions is convex (left), every convex shape is a limit of such maxima, and a ReLU input-convex network is a nested max-affine map.
- No CES, Cobb–Douglas or translog: it imposes the *shape*; the data choose within it.
- **The economist's usual position: theory pins the shape, not the curvature.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Y. Chen, Y. Shi, B. Zhang, "Optimal control via neural networks: a convex approach", *ICLR*, 2019. Figure: schematic.

What the Constraint Refuses, and Partial Convexity

- Point the same input-convex network at $\sin 3x$ on $[-1, 1]$: median RMSE 0.345, and the fit stays convex to within 9×10^{-11}. It cannot bend, which is the guarantee seen from the other side.
- **A structured network that cannot fit is saying the imposed theory is wrong**: an informative failure.
- A value function $V(k, z)$ is concave in the endogenous state k and of no particular shape in the shock z. Impose convexity on z too and you add bias that no residual reveals.
- **Partially input-convex** network: $z_{\ell+1} = \sigma\big(W_\ell^{(z)} z_\ell + W_\ell^{(u)}(u \circ v) + W_\ell^{(v)} v + b_\ell\big)$, $W_\ell^{(z)} \geq 0$; capital is the constrained input u, the shock the free input v.
- Concave V makes $\max_{k'}\{u(c) + \beta V(k')\}$ one basin, so gradient steps find its maximum (Session 3).
- **Impose the restriction exactly where theory gives one, and nowhere else.**

Source: Amos, Xu & Kolter, *ICML*, 2017, §3.2. $\sin 3x$: instructor's quantitative-macro course computations (2026), not recomputed here.

© Zhigang Feng · Redistribution prohibited without permission

Measured: What an Unconstrained Network Does to Convexity

	median RMSE	worst negative curvature
plain ReLU network	0.00172	5.93 (range 0 to 10.4 over seeds)
input-convex network	0.01345	below 7.1×10^{-11}

- Target: $0.6x^2 + 0.25x + 0.1$, exactly convex, curvature 1.2 everywhere. 1,153 parameters, five seeds.
- The plain network fits the *level* eight times better and is locally concave by about 5.9. Its loss shows none of that.
- At one seed in five it came out convex anyway: you cannot tell without checking.
- **The constrained network buys a guarantee and pays for it in fit: 7.8 times the RMSE.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

© Thiago Ferreira. Redistribution prohibited without permission

What Structure Buys, and What It Refuses

- **Buys reliability.** The shape holds at every θ, every seed, and outside the training band, where data cannot enforce it.
- **Buys well-behaved downstream problems.** Convex costs give monotone factor demands, so a market-clearing price is unique and a calibration loop converges.
- **Buys data efficiency.** Fewer functions to choose between means fewer states to pin one down.
- **Refuses flexibility.** If the true function violates the shape, the constraint is bias. It must come from theory, not convenience.
- Session 3 uses the same device for value functions.
- **Deck 2.5 imposed exactly one of these, feasibility, because it is the one theory guarantees on every rung.**

© Zigang Feng · Redistribution prohibited without permission

Session 2.6

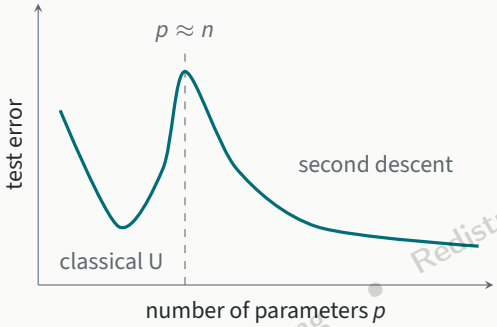
What More Parameters Buy

The opposite move: no constraint at all, and more parameters than observations



© Zhigang Feng Redistribution prohibited without permission

The Modern Twist: Double Descent

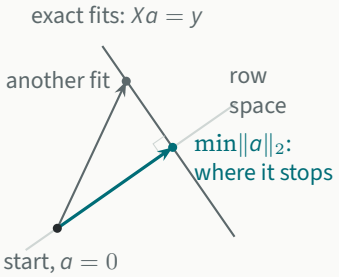


- Test error follows the textbook U, then peaks at the **interpolation threshold** $p \approx n$, where the model first fits the sample exactly.
- With $p \gg n$ it falls again: **double descent** (a schematic).
- In forecasting, $p > n$ can lower out-of-sample risk.
- **More parameters than observations is not a defect. It is a claim to test out of sample.**

© Zhigang Feng Redistribution prohibited without permission

Source: Measured on this deck's own example four frames on. M. Belkin, D. Hsu, S. Ma, S. Mandal, *PNAS* 116(32), 2019; Y. Liao, X. Ma, A. Neuhierl, Z. Shi, "Economic forecasts using many noises", arXiv 2312.05593, 2023.

Why Gradient Descent Picks That Fit

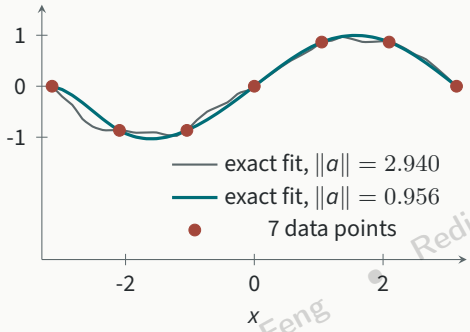


- Linear in the parameters: every gradient $\nabla_a L = X^\top(Xa - y)$, and so every step, lies in the **row space** of X.
- From $a = 0$ the iterate never leaves it, and one exact fit alone lies there: the smallest $\|a\|_2$. **The optimiser cannot move in directions the data never mention.**
- $\|\cdot\|_2$ is the selector, Euclidean because the steps are — not a smoothness claim. Multiplicative updates select ℓ_1 instead: the algorithm is part of the estimator.
- **With $p > n$ the algorithm picks the fit. Whether it generalises is empirical, not a theorem.**

© Zhigang Feng
Redistribution prohibited without permission

Source: The min-norm fact above is elementary: the derivation is its proof. S. Gunasekar, J. Lee, D. Soudry, N. Srebro, “Characterizing implicit bias in terms of optimization geometry”, ICML 2018.

Implicit Regularization, Stated Precisely

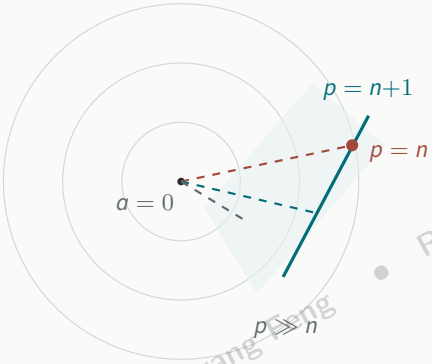


- Seven points of $\sin x$, 41 fixed features: the coefficients $a \in \mathbb{R}^{41}$ are not pinned down, and infinitely many fits are exact.
- Gradient descent from zero converges to the fit of **smallest Euclidean norm** (teal). No penalty was written.
- **The optimiser picks among exact fits. For deep networks: intuition, not a theorem.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block B3. Soudry et al., *JMLR* 19, 2018 (the classification analogue: the iterate's *direction* converges to the maximum margin); Hastie et al., *Ann. Statist.* 50(2), 2022 (risk at $p > n$).

Why the Norm Moves: It Is a Minimum over a Set



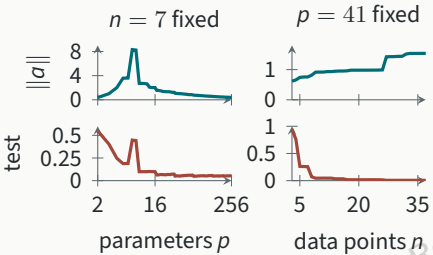
Dashed: the smallest member of each set.

- $\|a\|$ is the *smallest* coefficient vector that fits. It is a minimum over a set, so whatever changes the set changes it.
- **More parameters** give more ways to fit: the set grows, so its minimum falls. **More data** give more to satisfy: the set shrinks, so its minimum rises.
- At $p = n$ the set is a *single point* — nothing to choose — and that is the peak. Below $p = n$ nothing fits exactly at all.

Source: Both directions follow from one argument; both are checked in the Session 2 lab notebook, block B3.

© Zhirang Yang Redistribution prohibited without permission

Two Levers, Measured

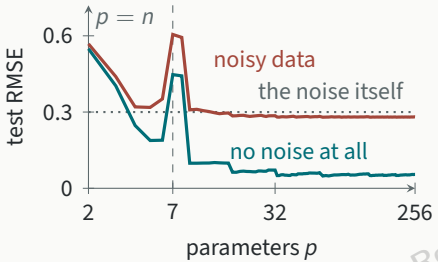


- **More** p (left): below $p = n$ nothing fits exactly; at $p = n$ the single fit costs 8.3; above it the norm falls to 0.4.
- **More** n (right): $0.62 \to 1.54$, and every fit is still exact.
- Test error follows neither: past $p = n$ it drops — the optimiser may now *choose* — then stalls near $p = 65$. More data never stops helping, $0.95 \to 0.006$.
- **The norm falls with the lever that need not help, and rises with the one that always does.**

Source: Session 2 lab notebook, block B3. Nested hinges and nested data points, so each sweep adds to what came before.

© Zhigang Feng
Redistribution prohibited without permission

Is That Just Overfitting?



- No: the training error is **zero for every $p \geq n$ on both curves**, so a loss below the noise floor cannot tell them apart.
- **Overfitting** drags the model onto noise. Here every candidate ties at zero; only the tie-break differs.
- With $y = \sin x$ exactly there is nothing to overfit, yet $p = n$ gives 0.45 against 0.19 at $p = 6$. With noise the two stack and the peak rises to 0.61.
- **Overfitting is cured with less capacity; this, with a better tie-break or more data.**

Source: Session 2 lab notebook, block B3; $n = 7$, noise s.d. 0.30, mean over 200 draws.

© Zhigang Feng
Redistribution prohibited without permission

Session 2.6

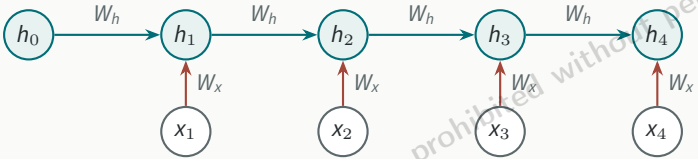
Order, and Its Absence

Sequences, and sets of agents where names do not matter



© Zhigang Feng Redistribution prohibited without permission

Recurrent Networks: the Hidden State Is a State Variable

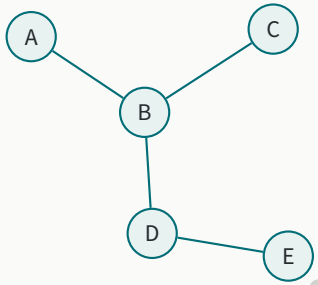


© Zhiguo Feng — Redistribution prohibited without permission

- A **recurrent network** reads a sequence one date at a time: $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$, the same W_h, W_x at every date.
- The **hidden state** h_t summarises the history: a state-space model with an estimated transition.
- Going back T dates multiplies by W_h^T, which vanishes or explodes. An LSTM's gates make persistence depend on the state; an AR(1) has a constant ρ.
- **The constraint: one law of motion at every date.**

Source: S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation* 9(8), 1997; Miao & Feng, §9.6.

Graph Networks: Permutation Invariance Is an Economic Restriction



five banks; an edge is lending

- A **graph network** updates each node from itself and its neighbours' average, with one pair of matrices for all nodes: $h'_v = \sigma\big(W_{\mathrm{s}} h_v + W_{\mathrm{n}}\, \overline{h}_{N(v)} + b\big)$.
- Neighbours have no order, so the aggregate is a sum or a mean: **permutation invariance**. Renumbering nodes cannot change the answer.
- **Heterogeneous-agent models impose the same: prices depend on the distribution, not on who is who (Session 4).**

© Zhigang Feng
Redistribution prohibited without permission

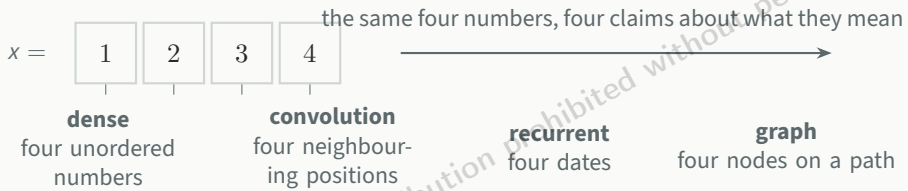
Source: T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks", ICLR 2017; Miao & Feng, §9.6.

Takeaways

- **An architecture is a prior about the input.** True, it saves parameters and data. False, it is bias.
- **A convolution is a dense layer after two constraints**: local, and the same filter everywhere.
- **Sign constraints on W buy a guarantee and pay for it in fit**: 7.8 times the RMSE on the convexity test, and a harder search — not a smaller class.
- **Shape is the constraint an economist can usually justify**, because theory delivers monotonicity and convexity before any data do.
- **More parameters than observations is not overfitting**: every candidate ties at zero training error, and only the tie-break differs.
- **Both halves of this deck are the same question: what decides the answer when the data do not?**

© Zhaoning Feng · Redistribution prohibited without permission

Appendix: One Input, Read Four Ways



© Zhiang Feng · Redistribution prohibited without permission

- Every layer below is the same $\sigma(Wx + b)$. They differ only in which entries of W are free, which are fixed at zero, and which are forced to be equal.
- Weights are chosen here, not trained, so every number on the next two pages can be checked by hand.
- **The input never changes. The architecture is the claim about what its four numbers are.**

Appendix: Four Layers, Four Answers

	Rule	Output	Free
dense	$w = (\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2})$	-1	5
convolution	filter $(1, -1)$, slid	$(-1, -1, -1)$	3
recurrent	$h_t = \frac{1}{2} h_{t-1} + x_t$	$(1,\ 2.5,\ 4.25,\ 6.125)$	3
graph	$h'_v = x_v + \frac{1}{2} \overline{x}_{N(v)}$	$(2,\ 3,\ 4.5,\ 5.5)$	3

1	−1	0	0
0	1	−1	0
0	0	1	−1

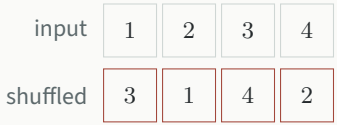
the convolution's W:
3×4, three numbers

- Dense returns one number, the convolution three, the recurrent and graph layers four: the constraint changes the *shape* of the answer, not only the count of parameters.
- The convolution's W is written out on the right. Nothing was added to the dense layer — entries were set to zero and the survivors tied together.
- **Three free numbers do the work of five, and only because the tying is true of the input.**

Source: Every value asserted against the arithmetic in the Session 2 lab notebook, appendix (one input, four architectures).

© Shigaev Pons
Redistribution prohibited without permission

Appendix: What Each One Refuses to Notice



convolution: $(-1, -1, -1)$

convolution: $(2, -3, 2)$

the dense layer, with its weights relabelled the same way, is unchanged

© Zhirang Feng Redistribution prohibited without permission

- **Shuffle the input.** The convolution's answer is destroyed: it assumed positions 1 and 2 were neighbours, and now they are not. The dense layer never made that claim, so it learns the shuffled problem just as well — the digit slide's point, as arithmetic.
- **Relabel the graph's nodes** *A B C D* as *D C B A*: the output becomes $(5.5,\ 4.5,\ 3,\ 2)$, the same four numbers in the new order, and a sum readout stays 15 either way. That is permutation invariance, as a number.
- The recurrent layer weights the past geometrically, so any reordering of the dates changes the answer: order is the whole of its claim.
- **A constraint is not a saving. It is a statement about the input that can be wrong.**